

I. Basic Computer Model

Assembly language is a low level programming language. Students need to get some knowledge about computer structure in order to understand anything about the assembler. The simple computer model is shown in figure 1.1:

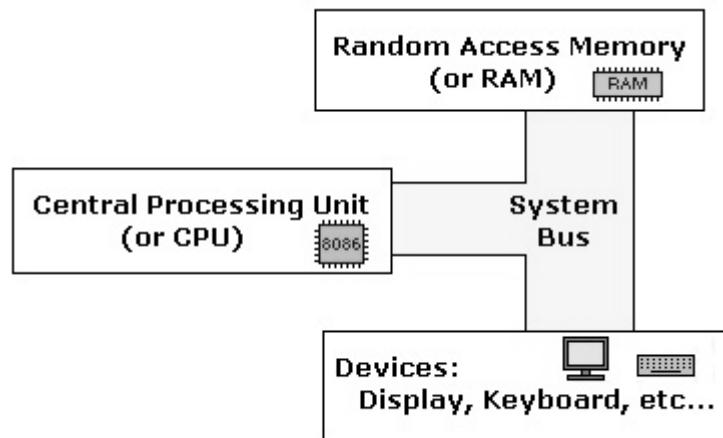


Figure 1.1 Simple Computer Model

The system bus connects the various components of a computer. The Central Processing Unit (CPU) is the heart of the computer, and most of the computations take place inside the CPU. Random Access Memory (RAM) is a temporary storage place where the programs are loaded in order to be executed.

II. Inside the Central Processing Unit (CPU)

The central processing unit is shown as the combination of registers in figure 1.2. The registers are discussed in the following sub-sections.

a) General Purpose Registers

The 8086/88 CPU has 8 general purpose registers, each register is discussed below:

- **AX** - the accumulator register (divided into AH / AL).
- **BX** - the base address register (divided into BH / BL).
- **CX** - the count register (divided into CH / CL).
- **DX** - the data register (divided into DH / DL).

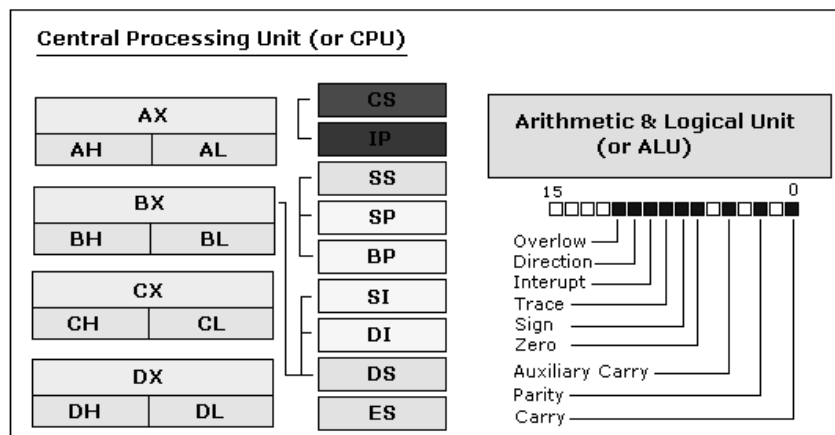


Figure 1.2 Central Processing Unit

Since registers are located inside the CPU, they manipulate data much faster than memory. Accessing data in a register usually takes no time. Accessing a memory location requires the use of a system bus, so it takes much longer. Therefore, it is recommended to keep data variables in registers. The main purpose of a register is to temporarily store a number, similar to a variable. The size of each register (listed above) is 16 bit. Four general purpose registers (AX, BX, CX, DX) can be split into two separate 8 bits (or one byte) registers, for example if $AX = 0011000000111001_2$, then $AH = 00110000_2$ and $AL = 00111001_2$. The same is applicable for other 3 registers (i.e., BX, CX, DX), where "H" is for higher and "L" is for lower byte.

b) Special Purpose Registers

The 8086/88 CPU has ten special purpose registers categorized in three different groups, namely, Index and pointer registers, Segment registers, Flag registers. Each is discussed below

i) Index and Pointer Registers

The 8086/88 CPU has two index and two pointer registers, as listed below:

- **SI - source index register.**
- **DI - destination index register.**
- **BP - base pointer.**
- **SP - stack pointer.**
- **IP - instruction pointer**

These registers are used for indexing an array of memory in association with the segment registers. For example, **IP** register always works together with **CS** segment register and it points to currently executing instruction in the program memory.

ii) Segment Registers

The 8086/88 CPU has 4 segment registers, each register is discussed below

- **CS - points at the segment containing the current program.**
- **DS - generally points at segment where variables are defined.**
- **ES - extra segment register, it's up to a coder to define its usage.**
- **SS - points at the segment containing the stack.**

Although it is possible to store any data in the segment registers, however, this is not recommended and is not considered as good practice. The segment registers have a very special purpose –to point at accessible blocks of memory.

Segment registers work together with general purpose registers, index registers and pointer registers to access any memory value. For example if we would like to access memory at the physical address **12345_h** (hexadecimal), we should set the **DS = 1230_h** and **SI = 0045_h**. This is called the logical address and this way much more memory can be accessed than using a single register that is limited to only 16 bit values. However, 8086/88 has accessible internal memory of 20bits (i.e., 2^{20} memory cells). For this, CPU makes a calculation of physical address by multiplying the segment register by 10_h (or 16₁₀) and adding general purpose register to the resulting value ($1230_h * 10_h + 45_h = 12345_h$):

$$\begin{array}{r} +12300 \\ +0045 \\ \hline 12345 \end{array}$$

The address formed with 2 registers, in such a way, is called an **effective address**.

By default **BX**, **SI** and **DI** registers work with **DS** segment register; **BP** and **SP** work with **SS** segment register. Other general purpose registers cannot be used to form an effective address. Moreover, only **BX** can form an effective address, **BH** and **BL** are not allowed for such purpose.

iii) Flag Registers

Flag registers are considered as special purpose registers. They indicate the current status of the microprocessor. Flag register is modified automatically by CPU after arithmetic and logical operations, this allows to determine the type of the result, and to determine conditions to transfer control to other parts of the program. Generally these registers cannot be accessed directly, similar to AX and other general registers. It is possible to change values of system registers using some recommended methods that will be discussed later in the course.

III. Number Systems

To understand the internal structure of any microprocessor and to learn the respective assembly language, the knowledge of number systems (specially, the hexadecimal number system) is necessary.

This lab provides some description and activities to refresh the concept of number systems.

Two number systems are considered to be used for the purpose of communication between human and machine. These number systems are **binary**, and **decimal**.

However, hexadecimal number systems is considered to be understandable by both human and machines.

In the following sub-sections, the decimal, binary, octal, and hexadecimal number systems are discussed briefly.

a) Decimal Number Systems:

It is also called Base-10 number system. Since it is base-10, therefore only ten items (i.e., 0, to 9) are used in this number system. These decimal (ten) items are termed as digits.

b) Binary number systems:

It is also called Base-2 number system. Since it is base-2, therefore only two digits (i.e., 0, and 1) are used in this number system. These binary digits are termed as bits (**binary digits**).

c) Octal Number Systems:

It is also called Base-8 number system. Since it is base-8, therefore only eight digits (i.e., 0, to 7) are used in this number system. These octal digits can also be represented as bits.

d) Hexadecimal Number Systems:

It is also called Base-16 number system. Since it is base-16, therefore only eight digits (i.e., 0, to 9, and A to B) are used in this number system. These hexadecimal digits can also be represented as bits.

IV. Conversion among Bases

The possibilities to convert among bases are shown in figure 1.3.

a) Conversion from decimal to binary

- Divide the given number by two, keeping track of the remainder
- First remainder is bit 0 with the weight 1 (LSB, least-Significant Bit)
- Second remainder is bit 1 with the weight 2
- Third remainder is bit 2 with the weight 4
- nth remainder is bit n-1 with the weight $2^{(n-1)}$ (MSB, Most-Significant Bit)

b) Conversion from decimal to octal

- Divide the given number by eight, keeping track of the remainder
- First remainder is bit 0 with the weight 1 (LSB, least-Significant Bit)
- Second remainder is bit 1
- Do the same till the last digit

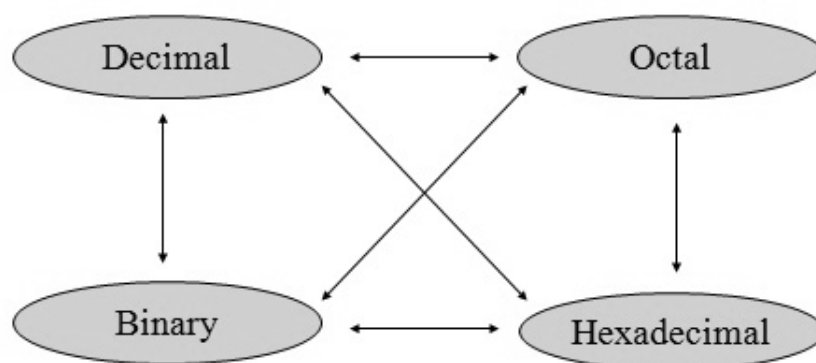


Figure 1.3 the possibilities to convert among bases

c) Conversion from decimal to hexadecimal

- Divide the given number by sixteen, keeping track of the remainder
- First remainder is bit 0 with the weight 1 (LSB, least-Significant Bit)
- Second remainder is bit 1
- Do the same till the last digit
-

d) Conversion from binary to decimal

- Multiply each bit by 2^n , the “weight” of the bit
- n is the sequence number of bits from LSB which is the right most bit
- Add the results

e) Conversion from binary to octal

- Group bits in threes, from right to left
- Convert to octal digits

f) Conversion from binary to hexadecimal

- Group bits in four, from right to left
- Convert to hexadecimal digits

g) Conversion from octal hexadecimal

- Use binary as an intermediary

h) Conversion from hexadecimal to octal

- Use binary as an intermediary

A Quick Example

Following is a quick example of such conversions.

$$25_{10} = \overset{1}{\overbrace{11001}^9}_3_2 = 31_8 = 19_{16}$$